MUTATION, CLASSES, & ATTRIBUTES Meta

COMPUTER SCIENCE MENTORS 61A

October 12, 2026 – October 16, 2026

Example Timeline

- **This is a menu of problems, not a requirement to finish the worksheet.** Choose or skip problems based on your students, and give each mini-lecture immediately before its section.
- Check-in and icebreaker [5 min]. Ask students for one operation that constructs a list and one that mutates a list.
- List mutation: selected parts of Q1 (WWPD), then Q2 (environment diagram) [15 min]. Track shallow copies, shared nested lists, and the return values of mutating methods. Q3, `insert_n`, is extra implementation practice.
- Classes and dataclasses: compare the two `Car` definitions, then selected parts of Q4 (museum tickets) [15 min]. Connect constructor arguments and defaults to instance attributes, then trace method calls, aliasing, and reassignment.
- Attributes and methods: selected parts of Q5 (`Account`) [10 min]. Distinguish functions from bound methods and trace dot expressions, attribute lookup, and assignment.
- Choose Q6 (counters) or Q7 (labels at a depth) [10 min]. Q6 practices saved bound methods and reassignment; draw the class and both instances. Q7 practices tree recursion and mutating a result list while leaving the input tree unchanged.
- Wrap-up and questions [5 min]. Ask students to explain mutation, attribute lookup, and bound methods in their own words.

Teaching Goals

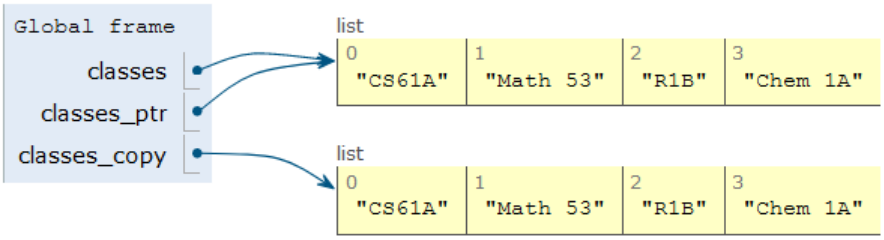
- Distinguish construction, reassignment, aliasing, and mutation. Mutation changes an existing object, so every reference to that object observes the change.
- Use box-and-pointer or environment diagrams to track list mutation, shallow copies, and shared nested objects. For instance mutation, draw attributes inside the instance that owns them.
- In Q7, each recursive call returns a list of labels at the requested depth. Use `extend` to combine those results, and return the list separately because `extend` returns `None`. Only the result lists are mutated, not the tree.
- Recognize that classes and instances are objects with attributes. Functions and bound methods are objects as well.
- Compare explicit and generated initializers for `Car`: both accept `owner` and optional `gas`; Python supplies `self`. The unannotated `wheels` remains a class attribute.
- Dataclasses support methods with parameters, return values, and mutation. In Q4, distinguish the boolean returned by `admit` from its change to `remaining`. Equal field values do not imply object identity.
- For Q5's ordinary attributes, lookup checks the instance and then its class. Assignment writes to the object before the dot: `a.interest = 7` shadows the class attribute; `Account.interest = 4` changes it.
- A function stored as a class attribute acts as a method. Looking it up through an instance produces a bound method: `a.deposit(5)` is equivalent to `Account.deposit(a, 5)`. In Q6, a saved bound method retains its instance even after the original name is reassigned.

Created by Cadence Loh, Ana Abell, Tien-Yu Hsu, and Rajoshi Basu. feedback form!
Provided by CSM 61A contributors under CC BY 4.0/3-clause BSD.

1 List Mutation and Environment Diagrams

Let's imagine it's your first year at Cal, and you have signed up for your first classes!

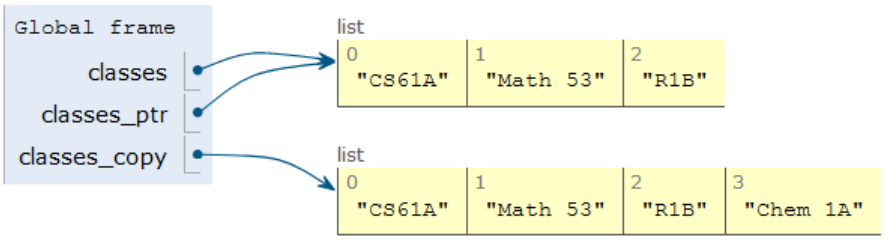
```
>>> classes = ["CS61A", "Math 53", "R1B", "Chem 1A"]
>>> classes_ptr = classes
>>> classes_copy = classes[:]
```



After a few weeks, you realize that you cannot keep up with the workload and you need to drop a class. You've chosen to drop Chem 1A. Based on what we know so far, to change our classes list, we would have to create a new list with all the same elements as the original list except for Chem 1A. But that is silly, since all we really need to do is remove the Chem 1A element from our list.

We can fix this issue with list mutation. In Python, some objects, such as lists and dictionaries, are mutable, meaning that their contents or state can be changed over the course of program execution. Other objects, such as numeric types, tuples, and strings are immutable, meaning they cannot be changed once they are created. Therefore, instead of creating a new list, we can just call `classes.pop()`, which removes the last element from the list.

```
>>> classes.pop() # pop returns whatever item it removed
"Chem 1A"
```



Here are more list methods that mutate:

Mutability in Lists

Function	Create/Mutate	Action	Return Value
lst.**append**(element)	mutate	attaches element to end of the list	None
lst.**extend**(iterable)	mutate	attaches each element in iterable to end of the list	None
lst **+=** lst2	mutate	attaches lst2 to the end of lst; same as lst.extend(lst2)	None
lst.**pop**()	mutate	removes last element from the list	Removed element
lst.**pop**(index)	mutate	removes element at index	Removed element
lst.**remove**(element)	mutate	removes element from the list	None
lst.**insert**(index, element)	mutate	inserts element at index and pushes rest of elements down	None
lst ***=** n	mutate	attaches lst to the end of lst (n - 1) times; same as calling lst.extend(lst) (n - 1) times	None
lst[start:end:step size]	create	creates a new list that includes elements from *start* index to *end* index - 1 while incrementing *step size*	new list
lst = lst **+** [1, 23]	create	creates a new list with elements from lst and [1, 23]	new list
lst = lst ***** n	create	creates a new list with elements from lst repeated n times	new list
list(iterable)	create	create new list with elements of iterable	new list

Teaching Tips

- Common Misconceptions:
  - Students may be confused about the return value of mutation functions.
    - Try contrasting `pop` with `remove`, showing them how only `pop` returns the element.
    - Tell them to reference the list mutability table. Come up and go over examples for each one of them so as to reinforce concepts before trying spooky out.
- The objectives for students are to:
  - Distinguish between mutable and non-mutable objects

- The effects and return values of mutation functions
- Become comfortable with pointers and how to copy objects

1. What would Python display? If an error occurs, write "Error". If a function is displayed, write "Function". If nothing is returned, write "Nothing". Also, draw box-and-pointer diagram for each list created.

```
>>> a = [1, 2]
>>> a[1]
```

```
2
```

```
>>> a.append([3, 4])
>>> a
```

```
[1, 2, [3, 4]]
```

```
>>> b = a[1:]
>>> a[1] = 5
>>> a[2][0] = 6
>>> b
```

```
[2, [6, 4]]
```

```
>>> a.extend([7])
>>> a += [8]
>>> a
```

```
[1, 5, [6, 4], 7, 8]
```

```
>>> a += 9
```

```
TypeError: 'int' object is not iterable
```

Box-and-pointer diagram: `https://goo.gl/YJfNgf`

Teaching Tips

1. Refer to above notes on box and pointer diagrams! When going through this one, draw the box and pointer diagrams on the board
2. Encourage students to draw box and pointer diagrams if they seem stuck
3. It can be helpful to visually go through indexing and list slicing in the box and pointer diagram
4. Python is confusing when adding on to a list with `lst += [element]` and `lst = lst + element`. The former mutates and the latter creates a new list
5. Make sure you clearly state when you're making a new list object (i.e. at `a = a + [4, [5, 6]]` and list slicing like `d = c[3:5]`)

6. Make sure to reiterate the difference between append and extend
7. Be sure to touch on negative indices and reiterate the difference between shallow and deep copies

2. Draw the environment diagram that results from running the following code.

```
a = [1, 2, [3]]

def mystery(s, t):
    s.pop(1)
    return t.append(s)

b = a
a += [b[0]]
a = mystery(b, a[1:])
```

https://goo.gl/s2XKiG

3. Write a function `insert_n`, which takes in an ascending list of numbers `lst`, a number `x`, and an integer `n`. If `n` is positive, `insert_n` mutatively inserts `n` copies of `x` into `lst` at the correct position so that `lst` is still in ascending order. If `n` is negative, `insert_n` mutatively removes `|n|` copies of `x` from `lst`. (Assume that there are always enough copies to `x` to be removed.)

```
def insert_n(lst, x, n):
    """
    >>> lst = []
    >>> insert_n(lst, 4, 1)
    >>> insert_n(lst, 1, 3)
    >>> insert_n(lst, 2, 2)
    >>> lst
    [1, 1, 1, 2, 2, 4]
    >>> insert_n(lst, 1, -2)
    >>> lst
    [1, 2, 2, 4]
    """
    if n > 0:
        i = 0
        while _____________________________________________:

            ____________________________________________

        ____________________________________________:

            ____________________________________________
    elif n < 0:

        ____________________________________________

            ____________________________________________
```

```
def insert_n(lst, x, n):
    """
    >>> lst = []
    >>> insert_n(lst, 4, 1)
    >>> insert_n(lst, 1, 3)
    >>> insert_n(lst, 2, 2)
    >>> lst
    [1, 1, 1, 2, 2, 4]
    >>> insert_n(lst, 1, -2)
    >>> lst
    [1, 2, 2, 4]
    """
    if n > 0:
        i = 0
        while i < len(lst) and lst[i] < x:
            i += 1
        for _ in range(n):
            lst.insert(i, x)
    elif n < 0:
        for _ in range(-n):
            lst.remove(x)
```

The purpose of this problem is to give students some hands on experience with list mutation methods, and also to give them a refresher on how to handle lists.

The trickiest part of this problem is figuring out where the elements need to be added. It is not sufficient to simply look for the earliest instance of `x` in `lst`, for example, because `lst` may not have any instances of `x` to start with. If students are stuck on this, you can try asking a series of leading questions to help them get there. For example: where do we want to insert our new elements? How can we insert our new elements into the correct place in the list? How can we figure out the correct index to insert our new elements? Why did they tell you that the list is in ascending order?

2 Classes and Instances

Object oriented programming is a programming paradigm that organizes relationships within data into **objects** and **classes**. In object oriented programming, each object is an **instance** of some particular class. For example, we can write a `Car` class that acts as a template for cars in general:

```
class Car:
    wheels = 4
    def __init__(self):
        self.gas = 100

    def drive(self):
        self.gas -= 10
        print("Current gas level:", self.gas)

my_car = Car()
```

To represent an individual car, we can then initialize a new instance of `Car` as `my_car` by "calling" the class. Doing so will automatically construct a new object of type `Car`, pass it into the `__init__` method (also called the **constructor**), and then return it. Often, the `__init__` method will initialize an object's **instance attributes**, variables specific to one object instead of all objects in its class. In this case, the `__init__` method initially sets the `gas` instance attribute of each car to 100. It is important to note, however, that you can also manually set object-specific attributes outside of the `__init__` method through variable declaration and methods.

Classes can also have **class attributes**, which are variables shared by all instances of a class. In the above example, `wheels` is shared by all instances of the `Car` class. In other words, all cars have 4 wheels.

Functions within classes are known as methods. **Instance methods** are special functions that act on the instances of a class. We've already seen the `__init__` method. We can call instance methods by using the dot notation we use for instance attributes:

```
>>> my_car.drive()
Current gas level: 90
```

In instance methods, `self` is the instance from which the method was called. We don't have to explicitly pass in `self` because, when we call an instance method from an instance, the instance is automatically passed into the first parameter of the method by Python. That is, `my_car.drive()` is exactly equivalent to the following:

```
>>> Car.drive(my_car)
Current gas level: 80
```

Something I like to emphasize with my students is that you can *only* access class and instance attributes using dot notation from an instance. That is, you can never just write `__init__` or `wheels`; you *must* use dot notation to access these attributes. The reason that students are confused by this is that the rules of variable scope in classes are different from those in functions. They often feel like because they are "inside" the class they should be able to access all of these variables without dot notation. I think it's often useful to dispel this notion by emphasizing that the rules are different and that it's essentially the objects and classes that "hold on" to their instance variables. But you should be careful when giving an explanation like this to not confuse your students more.

This overview is not meant to be a first exposure resource for your students, since there are so many ins and outs of OOP. It is likely that you will need to walk through some of the concepts in a more intuitive way than they are presented here.

The Same Data with a Dataclass

We can use the same `Car` example with either an explicit initializer or a dataclass. Here we extend the earlier example so construction accepts an owner and an optional starting gas level. These are alternative definitions; use one at a time.

```
class Car:
    wheels = 4

    def __init__(self, owner, gas=100):
        self.owner = owner
        self.gas = gas

    def drive(self):
        self.gas -= 10
        print("Current gas level:", self.gas)
```

A dataclass declares its instance attributes as annotated fields and generates an `__init__` method automatically. We can define methods in its class body exactly as before:

```
from dataclasses import dataclass

@dataclass
class Car:
    wheels = 4
    owner: str
    gas: int = 100

    def drive(self):
        self.gas -= 10
        print("Current gas level:", self.gas)
```

For this dataclass, the generated initializer takes `(self, owner, gas=100)` and assigns `self.owner` and `self.gas`, just like the explicit initializer above. The unannotated `wheels = 4` remains a class attribute, not a dataclass field or constructor argument. The default for `gas` lets us omit that argument.

The following works with either definition.

```
>>> my_car = Car("Ada")
>>> other_car = Car("Grace", 50)
>>> my_car.owner, my_car.gas, other_car.gas
('Ada', 100, 50)
>>> my_car.drive()
Current gas level: 90
>>> Car.drive(my_car)
Current gas level: 80
>>> other_car.gas
50
```

In either style, calling the class constructs an instance; Python supplies `self` automatically during initialization. Methods also work the same way: `my_car.drive` is a bound method, and `my_car.drive()` has the same effect as `Car.drive(my_car)`. A dataclass method can accept additional arguments, return a result, or mutate instance attributes. Here, `drive` changes the gas level of just the instance passed as `self`. The class object and each instance are objects with their own attributes. Dataclasses also generate conveniences such as a representation and field-based equality; they are still classes.

4. **Museum tickets.** Predict each result, executing the prompts in order. Draw the instances and the references to them as you trace.

```
from dataclasses import dataclass

@dataclass
class Ticket:
    exhibit: str
    remaining: int = 2

    def admit(self):
        if self.remaining > 0:
            self.remaining -= 1
            return True
        return False

original = Ticket("Space")
alias = original
separate = Ticket("Space", 1)


>>> original.remaining, separate.remaining

(2, 1)

>>> alias.admit()

True

>>> original.remaining, separate.remaining
>>> original is alias, original is separate


(1, 1)
(True, False)
```

Equal field values do not make two instances the same object.

```
>>> alias = Ticket("Ocean")
>>> alias.remaining = 5
>>> original.exhibit, original.remaining, alias.remaining
```

`('Space', 1, 5)`. Reassigning `alias` makes it refer to a new instance; the later attribute assignment changes that new instance.

```
>>> original.admit(), original.admit(), original.remaining

(True, False, 0)
```

Suggested Time: 8–10 min; Difficulty: Easy

- The generated initializer assigns the fields; methods are still ordinary function definitions in the class body.

- Draw two instances initially and a third when `alias` is reassigned. Track which object receives each attribute assignment.
- Ask students to rewrite `alias.admit()` as `Ticket.admit(alias)` and identify the value of `self`.

3 Attribute Lookup, Assignment, and Bound Methods

5. Consider only the following class; no inheritance is involved.

```
class Account:
    interest = 2

    def __init__(self, owner, balance=0):
        self.owner = owner
        self.balance = balance

    def deposit(self, amount):
        self.balance += amount
        return self.balance

a = Account("Ada", 10)
```

For each expression, write its value or the kind of object it produces. If an error occurs, write **Error**. Execute the expressions in order.

(a) `Account.interest, a.interest`

`(2, 2)`

(b) `Account.deposit`

A function object stored as a class attribute.

(c) `a.deposit`

A bound method object: `Account.deposit` with `a` already bound to `self`.

(d) `a.deposit(5)`

`15`

(e) `Account.deposit(a, 5)`

`20`

(f)
```
>>> a.interest = 7
>>> Account.interest = 4
>>> a.interest, Account("Bob").interest
```

`(7, 4)`. The first assignment writes an instance attribute on `a`; the second writes an attribute on the class object. The instance attribute shadows the class attribute during lookup on `a`.

(g) Explain how Python evaluates `a.deposit(10)`.

First evaluate `a.deposit`. Attribute lookup checks `a` and then `Account`, where it finds the function `deposit`. Looking up that function through an instance produces a bound method with `a` bound to its first parameter. Python then calls it with 10. The equivalent function call is:

`Account.deposit(a, 10)`

6. **Which counter changes?** Execute the prompts in order. Write each displayed value and draw separate attribute tables for the class and its two instances. If an expression raises an error, write **Error** and continue with the next prompt.

```
class Counter:
    step = 1

    def __init__(self, value):
        self.value = value

    def advance(self):
        self.value += self.step
        return self.value

left = Counter(0)
right = Counter(10)
saved = left.advance
```

(a)
```
>>> Counter.step = 3
>>> left.step = 2
>>> left.step, right.step, Counter.step
```

`(2, 3, 3)`. Only `left` has an instance attribute named `step`; `right` finds the class attribute.

(b)
```
>>> saved()
>>> Counter.advance(right)
```

```
2
13
```

The saved bound method supplies the original `left` instance as `self`. The class function requires an explicit instance.

(c)
```
>>> left = right
>>> saved()
>>> left.value, right.value
```

```
4
(13, 13)
```

Reassigning `left` does not change the instance held by `saved`. Both names now refer to the second counter, while the bound method still refers to the first.

Suggested Time: 8–10 min; Difficulty: Medium

- Draw a bound method as a reference to both a function and an instance. It retains the instance even if the name used to obtain it is reassigned.
- Track instance attributes separately from class attributes. Class lookup does not search its instances for their attributes.

4 Tree Recursion and Result Lists

For the following problems, use this definition for the `Tree` class:

```
class Tree:
    def __init__(self, label, branches=[]):
        self.label = label
        self.branches = list(branches)

    def is_leaf(self):
        return self.branches == []

    # Implementation ommitted
```

7. **Labels at a depth.** Implement `labels_at_depth`, which takes a tree `t` with integer labels and a nonnegative integer `k`. Return a new Python list containing the labels exactly `k` edges below the root, in left-to-right branch order. The root is at depth 0. If no nodes exist at that depth, return an empty list.

   Use recursion and `extend` to add each recursive result to the local `result` list. Do not change any tree labels or branches.

```
def labels_at_depth(t: Tree, k: int) -> list[int]:
    """
    >>> t = Tree(1, [Tree(2, [Tree(4), Tree(5)]),
    ...              Tree(3, [Tree(6)])])
    >>> labels_at_depth(t, 0)
    [1]
    >>> labels_at_depth(t, 1)
    [2, 3]
    >>> labels_at_depth(t, 2)
    [4, 5, 6]
    >>> labels_at_depth(t, 3)
    []
    >>> labels_at_depth(Tree(9), 1)
    []
    """
    if ____________________________________:

        return ____________________________

    result = []

    for b in ____________________________:

        result.extend(____________________)

    return result


def labels_at_depth(t: Tree, k: int) -> list[int]:
    if k == 0:
        return [t.label]
```

```
    result = []
    for b in t.branches:
        result.extend(labels_at_depth(b, k - 1))
    return result
```

At depth 0, return a one-element list containing the root's label. Otherwise, the requested nodes are at depth `k - 1` within each branch. Each call creates its own result list. `extend` mutates that list, but the input tree is only read. At a leaf with positive `k`, the loop has no iterations, so the result is empty.

Explain what would change if `extend` were replaced by `append`. Why would **`return`** `result.extend(...)` be wrong?

`append` adds each recursive result as a single list element, introducing nested lists. For example, at depth 1 the result would be `[[2], [3]]`. `extend` adds the elements individually. It returns `None`, so we must return `result` separately.

Suggested Time: 10 min; Difficulty: Medium

- Ask what the recursive call returns before filling in the loop. Decreasing `k` measures the remaining distance, not a label value.
- Draw a fresh result list for each recursive call. Mutation occurs in those lists; no assignment targets a tree attribute or its branches list.
- Trace an uneven tree or a depth beyond the leaves to explain why a separate leaf base case is unnecessary.